Angular Performance — v12 vs v18

Real-World Examples for Every Optimisation

> A side-by-side comparison of all 9 performance patterns across Angular 12 and Angular 18, with production-grade code examples and real-world impact notes.

Table of Contents

1. Change Detection Strategy
2. TrackBy — @for control flow
3. Lazy Loading
4. Async Pipe and toSignal
5. Pure Pipes vs Template Methods
6. Virtual Scrolling
7. Signals — Reactive State
8. HTTP Caching
9. @defer — Template-Level Lazy Loading
10. Version Comparison Summary

---

1. Change Detection Strategy

Status: Improved in v18 — Signals make this almost automatic.

Angular checks for UI changes by running change detection. The default strategy checks every component on every browser event. `OnPush` limits checks to when `@Input()` references change. v18 Signals make this granular automatically.

Angular v12 — Manual OnPush

```
// You had to set this on every component manually

// and understand reference equality to avoid stale UI

@Component({

  selector: 'app-expense-card',

  changeDetection: ChangeDetectionStrategy.OnPush,

  template: `

    <div class="card">

      <h3>{{ expense.title }}</h3>

      <p>{{ expense.amount | currency:'GBP' }}</p>

    </div>

  `

})

export class ExpenseCardComponent {

  @Input() expense: Expense;



  // ⚠️  Must pass a NEW object to trigger update

  // this.expense.amount = 50      → NO update (same reference)

  // this.expense = {...e, amount: 50} → updates (new reference)

}
```

Angular v18 — Signals, Automatic Tracking

```
// No changeDetection property needed

// Signal reads automatically register as dependencies

@Component({

  selector: 'app-expense-card',

  template: `

    <div class="card">

      <h3>{{ expense().title }}</h3>

      <p>{{ expense().amount | currency:'GBP' }}</p>

    </div>

  `

})

export class ExpenseCardComponent {

  expense = input<Expense>();

  // Signal input — Angular tracks exactly which

  // DOM expressions depend on this signal.

  // Only those expressions re-render when it changes.

}



// Enable zoneless mode in app.config.ts
```

```
export const appConfig: ApplicationConfig = {

  providers: [

    provideExperimentalZonelessChangeDetection()

    // Removes zone.js entirely — zero overhead

  ]

};
```

> **Real-world impact:** An expense dashboard with 200 cards. v12 OnPush checks all 200 components when anything in the app changes. v18 Signals update only the single DOM expression inside the one card whose data actually changed.

---

2. TrackBy / @for Control Flow

Status: Improved in v18 — `track` is now required by the compiler.

Without `trackBy`, Angular destroys and recreates every DOM node in a list when the list changes. In v12 this was optional. In v18's `@for` syntax, `track` is required — omitting it is a compile error.

Angular v12 — *ngFor with manual trackBy

```
// Template

<div *ngFor="let order of orders; trackBy: trackByOrderId">

  <app-order-card [order]="order"></app-order-card>

</div>
```

```
// Component

trackByOrderId(index: number, order: Order): number {

  return order.id;

  // Easy to forget this function entirely.

  // Forget it → full DOM rebuild on every change.

}
```

Angular v18 — @for, track required

```
<!-- track is part of the syntax — cannot be omitted -->

@for (order of orders; track order.id) {

  <app-order-card [order]="order" />

}




<!-- Bonus: @empty block for zero-state UI -->

@for (order of orders; track order.id) {

  <app-order-card [order]="order" />

} @empty {

  <p class="empty-state">No orders found.</p>

}
```

> **Real-world impact:** An order list with 100 items. User sorts by date. - **v12 without trackBy:** 100 destroy + 100 create = 200 DOM operations. - **v18 @for:** 0 destroy, 0 create — Angular

> reorders existing nodes in place.

---

3. Lazy Loading

Status: Improved in v18 — NgModule boilerplate eliminated.

Lazy loading defers the download of a feature's JavaScript bundle until the user navigates to that route. v12 required a dedicated NgModule for every lazy-loaded route. v18 loads a standalone component directly.

Angular v12 — Module-based lazy loading

```
// app-routing.module.ts

const routes: Routes = [

  {

    path: 'reports',

    loadChildren: () =>

      import('./reports/reports.module').then(m => m.ReportsModule)

  }

];



// reports/reports.module.ts — required boilerplate

@NgModule({

  declarations: [

    ReportsComponent,
```

```
    ChartComponent,

    FilterComponent

  ],

  imports: [CommonModule, RouterModule.forChild(reportRoutes)],

})

export class ReportsModule {}

// A whole module file just to lazy-load one route.
```

Angular v18 — Standalone, no module needed

```
// app.routes.ts

export const routes: Routes = [

  {

    path: 'reports',

    loadComponent: () =>

      import('./reports/reports.component')

        .then(m => m.ReportsComponent)

    // Points directly at the component — no module needed.

  }

];



// reports.component.ts
```

```
@Component({

  standalone: true,

  imports: [CommonModule, ChartComponent, FilterComponent],

  template: `

    <app-chart />

    <app-filter />

  `

})

export class ReportsComponent {}

// Component declares its own dependencies — self-contained.
```

> **Real-world impact:** A banking app with 8 feature routes. With lazy loading, the initial bundle is ~80 KB instead of ~400 KB. First paint is 4× faster for new users.

---

4. Async Pipe and toSignal

Status: Same concept, enhanced in v18 with `toSignal()`.

Manual subscriptions risk memory leaks if `unsubscribe()` is forgotten. The `async` pipe handles this automatically in both versions. v18 adds `toSignal()` which removes the need for `| async` in templates entirely.

Angular v12 — async pipe

```
// ❌ Manual subscription — memory leak risk

export class DashboardComponent implements OnInit, OnDestroy {
```

```
  transactions: Transaction[] = [];

  private sub = new Subscription();



  ngOnInit() {

    this.sub = this.txService.getTransactions()

      .subscribe(t => this.transactions = t);

  }



  ngOnDestroy() {

    this.sub.unsubscribe(); // Easy to forget!

  }

}



// ✅ async pipe — auto-unsubscribes, works with OnPush

export class DashboardComponent {

  transactions$ = this.txService.getTransactions();

}

<!-- Template -->

<div *ngFor="let t of transactions$ | async; trackBy: trackById">

  <app-tx-row [tx]="t"></app-tx-row>
```

```
</div>
```

Angular v18 — toSignal()

```
import { toSignal } from '@angular/core/rxjs-interop';



export class DashboardComponent {

  // Converts Observable → Signal

  // Auto-unsubscribes when component is destroyed.

  // No async pipe needed in the template.

  transactions = toSignal(

    this.txService.getTransactions(),

    { initialValue: [] as Transaction[] }

  );

}

<!-- Template — no | async needed -->

@for (t of transactions(); track t.id) {

  <app-tx-row [tx]="t" />

}
```

> **Key difference:** `toSignal()` integrates Observables into the signal graph, giving fine-grained reactivity without changing your RxJS service code.

---

5. Pure Pipes vs Template Methods

Status: Same in both versions — a fundamental Angular best practice.

Methods called inside templates execute on every change detection cycle. Pure pipes are cached and only recompute when their input reference changes.

Both versions — method in template (avoid)

```
// ❌ formatCurrency() called on EVERY change detection cycle

// In a table of 50 transactions this runs hundreds of times per second.



// Template

<td>{{ formatCurrency(tx.amount) }}</td>

<td>{{ getStatusLabel(tx.status) }}</td>



// Component

formatCurrency(amount: number): string {

  // Creates a new Intl object on every call — wasteful!

  return new Intl.NumberFormat('en-GB', {

    style: 'currency',

    currency: 'GBP'

  }).format(amount);

}
```

Both versions — pure pipe (recommended)

```
// ✅ Only recalculates when tx.amount changes reference



@Pipe({ name: 'gbp', pure: true })

export class GbpPipe implements PipeTransform {



  // Formatter created once — reused on every transform call

  private formatter = new Intl.NumberFormat('en-GB', {

    style: 'currency',

    currency: 'GBP'

  });



  transform(value: number): string {

    return this.formatter.format(value);

  }

}

<!-- Template -->

<td>{{ tx.amount | gbp }}</td>

<td>{{ tx.status | statusLabel }}</td>
```

> **Real-world impact:** A transaction table with 50 rows. Template method version runs 100 calls per CD cycle. Pure pipe version runs 0 calls when no values changed — potentially saving

> thousands of wasted computations per second on an active screen.

---

6. Virtual Scrolling

Status: Same CDK API in both versions. Setup is simpler in v18 (standalone import).

Without virtual scrolling, rendering 10,000 list items creates 10,000 DOM nodes on load. Virtual scroll renders only what is visible in the viewport — typically ~15 nodes regardless of list size.

Angular v12 — regular *ngFor

```
<!-- Renders ALL 10,000 rows on load -->

<div class="list-container">

  <div *ngFor="let txn of transactions; trackBy: trackById">

    <div class="row">

      <span>{{ txn.date | date }}</span>

      <span>{{ txn.description }}</span>

      <span>{{ txn.amount | currency:'GBP' }}</span>

    </div>

  </div>

</div>

<!-- 10,000 DOM nodes → 3-4 second render → janky scroll -->
```

v12 and v18 — CDK virtual scroll

```
// v12: import in NgModule

@NgModule({

  imports: [ScrollingModule]

})

export class TransactionsModule {}




// v18: import directly in standalone component

@Component({

  standalone: true,

  imports: [ScrollingModule, AsyncPipe],

  template: `...`

})

export class TransactionsComponent {}

<!-- Only ~15 DOM nodes exist at any time -->

<cdk-virtual-scroll-viewport itemSize="56" style="height: 500px;">

  <div *cdkVirtualFor="let txn of transactions$; trackBy: trackById">

    <div class="row">

      <span>{{ txn.date | date }}</span>

      <span>{{ txn.description }}</span>
```

```
      <span>{{ txn.amount | currency:'GBP' }}</span>

    </div>

  </div>

</cdk-virtual-scroll-viewport>

<!-- 10,000 items → instant render → smooth 60fps scroll -->
```

> **Real-world impact:** A bank statement with 3 years of daily transactions (~1,095 rows). Regular ngFor: ~4 second render, sluggish scroll. Virtual scroll: instant load, 60fps scroll. `itemSize` must match your actual row height in pixels.

---

7. Signals — Reactive State

Status: New in v16 (developer preview), stable in v17+, fully recommended in v18.

Signals replace `BehaviorSubject` for local component state. They are synchronous, require no subscriptions, and Angular uses them for fine-grained DOM updates.

Angular v12 — BehaviorSubject pattern

```
export class CartComponent implements OnDestroy {

  private items$ = new BehaviorSubject<CartItem[]>([]);

  private sub = new Subscription();



  // Derived value — must pipe from the subject

  total$ = this.items$.pipe(
```

```
    map(items => items.reduce((sum, i) => sum + i.price, 0))

  );



  addItem(item: CartItem) {

    const current = this.items$.value;

    this.items$.next([...current, item]); // Must call .next()

  }



  ngOnDestroy() {

    this.sub.unsubscribe(); // Must remember to clean up

  }

}

<!-- Template — async pipe required -->

<p>Items: {{ (items$ | async)?.length }}</p>

<p>Total: {{ (total$ | async) | currency:'GBP' }}</p>
```

Angular v18 — Signals

```
export class CartComponent {

  // Writable signal — no Subject, no Observable

  items = signal<CartItem[]>([]);
```

```
  // Computed signal — auto-recalculates when items changes
  total = computed(() =>
    this.items().reduce((sum, i) => sum + i.price, 0)
  );


  addItem(item: CartItem) {
    this.items.update(current => [...current, item]);
    // No .next(), no subscriptions, no ngOnDestroy needed
  }


  // Side effects with effect()
  constructor() {
    effect(() => {
      console.log('Cart updated:', this.items().length, 'items');
      // Runs whenever items signal changes
    });
  }
}
<!-- Template — no async pipe, just call the signal -->
```

```
<p>Items: {{ items().length }}</p>

<p>Total: {{ total() | currency:'GBP' }}</p>
```

> **Why Signals win:** BehaviorSubject is powerful RxJS but complex — subscriptions to create, pipes to chain, unsubscribe to manage, async pipe in templates. Signals are simpler, synchronous, and give Angular precise information about which DOM node changed so it can skip all others.

---

8. HTTP Caching with shareReplay

Status: Same RxJS pattern in both. v18 improves the interceptor API.

Prevent the same endpoint being called multiple times when several components subscribe simultaneously. In a dashboard where 3 widgets all call `getAccounts()`, without caching you get 3 HTTP requests on load.

Angular v12 — shareReplay in service

```
@Injectable({ providedIn: 'root' })

export class AccountService {

  private accounts$: Observable<Account[]>;



  constructor(private http: HttpClient) {}



  getAccounts(): Observable<Account[]> {

    if (!this.accounts$) {
```

```
      this.accounts$ = this.http

        .get<Account[]>('/api/accounts')

        .pipe(

          shareReplay(1)

          // All subscribers share one HTTP call.

          // Late subscribers immediately receive the cached result.

        );

    }

    return this.accounts$;

  }

}
```

Angular v18 — Functional HTTP interceptor

```
// cache.service.ts

@Injectable({ providedIn: 'root' })

export class CacheService {

  private store = new Map<string, HttpResponse<unknown>>();

  get(url: string)  { return this.store.get(url); }

  set(url: string, response: HttpResponse<unknown>) {

    this.store.set(url, response);

  }
```

```
}



// cache.interceptor.ts — functional interceptor (no class needed)

export const cacheInterceptor: HttpInterceptorFn = (req, next) => {

  if (req.method !== 'GET') return next(req); // only cache GETs



  const cache = inject(CacheService);

  const cached = cache.get(req.url);

  if (cached) return of(cached);



  return next(req).pipe(

    tap(event => {

      if (event instanceof HttpResponse) {

        cache.set(req.url, event);

      }

    })

  );

};



// app.config.ts — wire it up
```

```
export const appConfig: ApplicationConfig = {

  providers: [

    provideHttpClient(

      withInterceptors([cacheInterceptor])

    )

  ]

};
```

> **Real-world impact:** Dashboard with 3 child components all calling `getAccounts()`. Without caching: 3 HTTP calls on load, components render at slightly different times causing layout flicker. With caching: 1 call, all 3 components render from the same response simultaneously.

---

9. @defer — Template-Level Lazy Loading

Status: New in Angular v17, stable and recommended in v18. No v12 equivalent.

`@defer` lazily loads a component — including its JavaScript bundle — at the template level, not just at the route level. The bundle is only downloaded when the trigger condition is met.

Angular v12 — no template-level lazy load

```
<!-- All components and their bundles load on page load -->

<app-header></app-header>

<app-hero-section></app-hero-section>
```

```
<!-- Heavy charting library (Chart.js = 200KB) — always downloaded -->

<app-analytics-chart [data]="chartData"></app-analytics-chart>



<app-comments-section></app-comments-section>

<app-footer></app-footer>



<!-- Only option in v12: route-level lazy loading.

     Cannot defer individual components within a page. -->
```

Angular v18 — @defer with triggers

```
<app-header />

<app-hero-section />



<!-- Chart bundle downloads only when this enters the viewport -->

@defer (on viewport) {

  <app-analytics-chart [data]="chartData" />



} @loading (minimum 300ms) {

  <!-- Shown while the bundle is downloading -->

  <app-chart-skeleton />
```

```
} @placeholder {
  <!-- Shown before the trigger fires -->
  <div class="chart-placeholder" style="height: 400px;"></div>

} @error {
  <!-- Shown if the dynamic import fails -->
  <p>Chart could not be loaded. Please refresh.</p>
}
```

All @defer trigger types

```
<!-- Load when component enters the viewport -->
@defer (on viewport) { <app-heavy-map /> }

<!-- Load on first user interaction (click, keypress, etc.) -->
@defer (on interaction) { <app-rich-editor /> }

<!-- Load when the browser is idle (requestIdleCallback) -->
@defer (on idle) { <app-recommendations /> }
```

```
<!-- Load after a fixed delay -->

@defer (on timer(2000)) { <app-chat-widget /> }



<!-- Load when a Signal or expression becomes true -->

@defer (when userIsLoggedIn()) { <app-account-panel /> }



<!-- Prefetch the bundle early, render on a different trigger -->

@defer (on viewport; prefetch on idle) {

  <app-analytics-chart />

}
```

> **Real-world impact:** A reporting page with a heavy charting library (Chart.js ≈ 200 KB). Without `@defer`: every user downloads 200 KB on page load even if they never scroll to the chart. With `@defer (on viewport)`: only users who scroll to the chart download it. Initial page 40–60% smaller. LCP (Largest Contentful Paint) score improves significantly.

---

10. Version Comparison Summary

Feature status at a glance

Feature	Angular v12	Angular v18	Change
Change detection	Manual `OnPush` required	Signals + zoneless mode	Improved
List rendering	`*ngFor` + optional trackBy	`@for` with required `track`	Improved
Lazy loading	Module-based `loadChildren`	Standalone `loadComponent`	Improved

Feature	Angular v12	Angular v18	Change
Subscriptions	async pipe or manual	`toSignal()` or async pipe	Improved
Pure pipes	Available, manual setup	Same — no change	Same
Virtual scroll	CDK `ScrollingModule`	Same CDK, standalone import	Same
Local state	`BehaviorSubject` + RxJS	`signal()` + `computed()`	New
HTTP interceptors	Class-based	Functional `HttpInterceptorFn`	Improved
Template lazy load	Not available	`@defer` with triggers	New
Control flow	`*ngIf`, `*ngFor`, `*ngSwitch`	`@if`, `@for`, `@switch`, `@defer`	New

Migration priority guide

Priority	Action	Impact
High	Add `track` to every `@for` (or `trackBy` to every `*ngFor`)	Prevents full list DOM rebuilds
High	Apply `ChangeDetectionStrategy.OnPush` to all components	Reduces CD cycles by 60–80%
High	Lazy-load all feature routes	Cuts initial bundle size
Medium	Replace template method calls with pure pipes	Eliminates wasted computations
Medium	Migrate BehaviorSubject to `signal()` for local state	Simpler code, better performance
Medium	Use `@defer` for heavy below-the-fold components	Improves LCP and TTI
Medium	Add `shareReplay(1)` or a caching interceptor	Prevents duplicate HTTP calls
Lower	Enable CDK virtual scroll on lists over 200 items	Essential for large datasets
Lower	Enable zoneless change detection	Maximum performance, no zone.js overhead

---

Key Angular v18 concepts to know for interviews

Signals are synchronous reactive primitives. `signal(value)` creates a writable signal. `computed(() => ...)` creates a derived signal. `effect(() => ...)` runs a side effect when dependencies change. Angular uses signals for fine-grained DOM updates — only the expressions that read a changed signal re-render.

Standalone components have no NgModule. They declare their own imports array. Enabled by `standalone: true` in `@Component`. Now the default when creating new components with the Angular CLI.

New control flow (`@if`, `@for`, `@switch`, `@defer`) replaces structural directives (`*ngIf`, `*ngFor`). It is built into the template compiler — no import needed, better type narrowing, required `track` in `@for`, and `@defer` for lazy loading.

Zoneless change detection removes zone.js entirely. Change detection only runs when a signal changes or `markForCheck()` is called. Opt in with `provideExperimentalZonelessChangeDetection()`.

These examples are based on a real expense tracker / banking dashboard app. All patterns apply to any Angular project of similar scale.